

Informe de Laboratorio 05

Tema: Base de Datos

Nota

Estudiantes	Escuela	Asignatura
Gustavo Alonso Carrillo Villalta Fredy José Aragón Carpio Diego Marcelo Arce Coaquira José Manuel Bravo Rojas	Escuela Profesional de Ingeniería de Sistemas	Desarrollo de Aplicaciones Web Semestre: I Código: 2025004 Grupo: Desarrolladores

Laboratorio	Tema	Duración
05	Base de Datos	04 horas

Semestre académico	Fecha de inicio	Fecha de entrega
2026 - A	05 de Mayo de 2026	12 de Mayo de 2026

1. Laboratorio 05: Base de datos

En este laboratorio se elaboró el modelo lógico DER y su implementación física en PostgreSQL, siguiendo estándares de nomenclatura en inglés, con tablas en plural y camelCase, incluyendo los atributos obligatorios `id`, `status`, `created`, `modified`, `created_id` y `modified_id`.

El modelo fue implementado en **Postgre**

Para la documentación, se elaboró un archivo `README.md` en el repositorio de GitHub que detalla el modelo DER, el modelo físico y los anexos correspondientes.

1.1. Descripción y Objetivos

Se diseñó un modelo de base de datos escalable y profesional para un negocio de cafetería, siguiendo estándares modernos de desarrollo.

- Diseñar el modelo lógico DER del sistema CafeSys.
- Implementar el modelo físico en PostgreSQL utilizando UUID, auditoría y soft delete.
- Organizar las tablas por módulos funcionales con código de colores.
- Insertar datos de prueba para validar la integridad referencial.

- Preparar la base de datos para su futura integración con Django.
- Documentar el proyecto en GitHub.

Se creó una base de datos robusta para CafeSys con 22 tablas, cubriendo catálogo de productos, inventario de ingredientes, gestión de pedidos, delivery, ubigeo y producción. Se utilizó PostgreSQL local y se generaron scripts ordenados de creación e inserción de datos.

Como parte de las buenas prácticas, se elaboró un archivo **README.md** en el repositorio de GitHub que documenta el modelo DER, el modelo físico, la implementación en Supabase y los anexos correspondientes.

1.2. Resumen del Ejercicio

En este ejercicio se elaboró el modelo lógico DER y su implementación. La base de datos fue implementada en **Postgree** para una base de datos relacional de una cafetería, siguiendo estándares de nomenclatura en inglés con tablas en plural y camelCase.

Finalmente, se documentó todo el proceso en un archivo **README.md** en el repositorio de GitHub, incluyendo el modelo DER, los scripts SQL y las capturas de la implementación en Supabase.

1.3. Entregables

El proyecto se documenta y entrega a través de:

- **URL del Repositorio:** <https://github.com/FredyAragon/Cafe-Sys/tree/main>
- **Informe:** <https://github.com/FredyAragon/Cafe-Sys/blob/main/Informes/Informe-Lab5-DAW.pdf>

1.4. Entorno de Implementación

Para la implementación de la base de datos se utilizaron las siguientes herramientas: PostgreSQL como motor de base de datos relacional, Supabase como plataforma BaaS en la nube, y el editor de consultas SQL integrado en Supabase para la ejecución de los scripts.

1.4.1. Acceso a Supabase

Se accede al panel de Supabase mediante:

- **Panel Supabase:** <https://zdogupggtajcxuwrfrvr.supabase.co>
- **Editor SQL:** Project → SQL Editor → New Query

2. Equipos, materiales y temas utilizados

- Sistema Operativo: Windows 11
- PostgreSQL como motor de base de datos relacional
- Supabase como plataforma BaaS en la nube
- Editor SQL integrado en Supabase
- Visual Studio Code
- Git y GitHub
- dbdiagram.io para la elaboración del modelo DER
- Draw.io / Lucidchart para diagramas adicionales

3. URL del Repositorio en GitHub

El ejercicio desarrollado en el presente laboratorio se encuentra disponible en el repositorio oficial alojado en GitHub, perteneciente a CafeSys. A continuación, se presenta el enlace al repositorio y las rutas de acceso al proyecto:

- **URL del Repositorio:** <https://github.com/FredyAragon/Cafe-Sys.git>

4. Desarrollo de actividades del laboratorio

4.1. Estructura del Proyecto

El proyecto está organizado de la siguiente manera:

```
BD/
|----README.md
|----CafeSys_DB.sql
|----.gitignore

|----Informes/
|       +----Informe-Lab5-DAW.pdf

|----backend/
|       |----api/
|       |----config/
|       |----manage.py
|       +----requirements.txt

+----webapp/
|       +----__pycache__/
|       |----controllers.cpython-312.pyc
|       +----databases.cpython-312.pyc
```

4.2. Resolución del Ejercicio

4.2.1. Arquitectura de la solución

La base de datos fue diseñada siguiendo el modelo relacional, estructurando las entidades en tablas normalizadas con sus respectivas claves primarias, foráneas y restricciones. El diseño se organizó en cuatro versiones incrementales, partiendo de un núcleo base de 11 tablas hasta alcanzar 22 tablas con módulos de envíos, ubigeo y recetas.

4.2.2. Lógica del modelo

El flujo del diseño se describe de la siguiente manera:

- Se identificaron las entidades principales del sistema (usuarios, productos, pedidos) y sus relaciones.
- Se elaboró el modelo lógico DER representando entidades, atributos y relaciones N:M, 1:N y 1:1.
- Se implementó el modelo físico en PostgreSQL aplicando las convenciones de nomenclatura requeridas.
- Las tablas intermedias producto de relaciones N:M fueron nombradas en orden alfabético.
- Se insertaron registros de prueba para validar la integridad referencial del esquema.
- Finalmente, se desplegó la base de datos en Supabase y se verificó su correcta implementación.

5. An lisis del Modelo

5.1. Modelo L gico DER

El modelo entidad-relaci n (DER) representa la estructura conceptual de la base de datos del sistema de cafeter a. En  l se identifican 22 entidades, incluyendo usuarios, roles, productos, categor as, pedidos, detalles de pedidos, promociones e inventario, as  como sus relaciones de tipo 1:N y N:M.

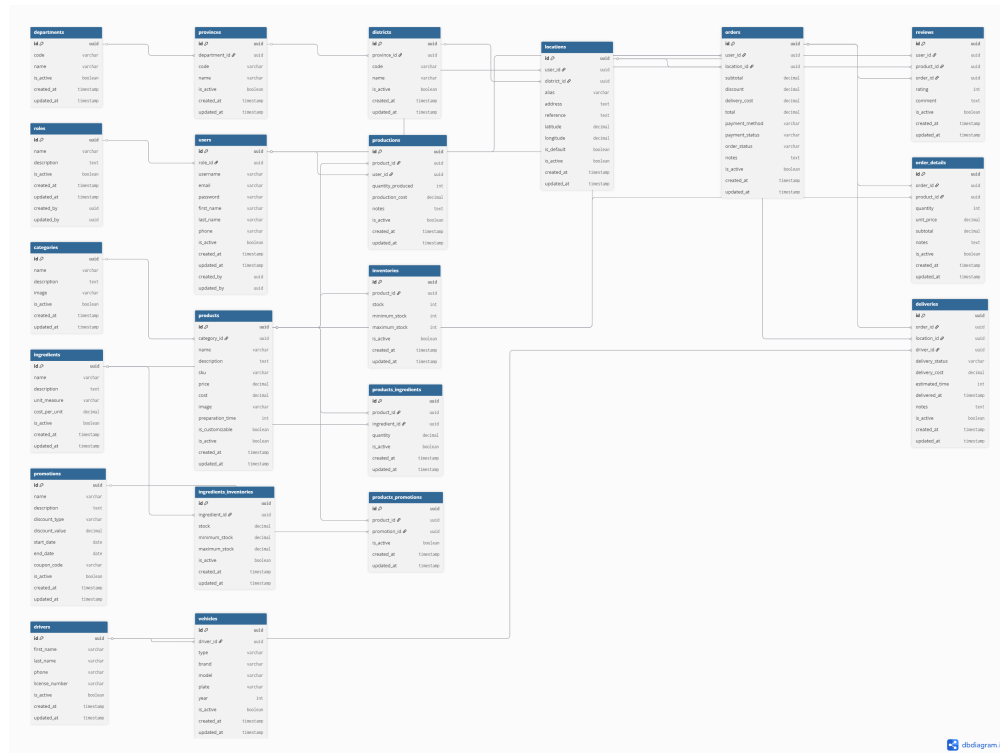


Figura 1: Modelo l gico DER del sistema de cafeter a con sus 22 tablas y relaciones.

5.2. Modelo F sico — Scripts SQL

Los scripts fueron generados respetando el orden de dependencias y buenas pr cticas profesionales (UUID como clave primaria, auditor a completa, soft delete con `is_active` y restricciones CHECK).

A continuaci n se muestran las tablas m s representativas organizadas por m dulos con su color correspondiente:

5.2.1. M dulo Usuarios y Seguridad (Azul)

Listing 1: Tablas roles y users

```
CREATE TABLE roles (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  name VARCHAR(50) NOT NULL UNIQUE,
  description TEXT,
  is_active BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  created_by UUID,
  updated_by UUID
```

```
);  
  
CREATE TABLE users (  
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
    role_id UUID NOT NULL REFERENCES roles(id),  
    username VARCHAR(150) UNIQUE NOT NULL,  
    email VARCHAR(255) UNIQUE NOT NULL,  
    first_name VARCHAR(100),  
    last_name VARCHAR(100),  
    phone VARCHAR(20),  
    is_active BOOLEAN DEFAULT TRUE  
);
```

5.2.2. M dulo Ubigeo y Direcciones (Morado)

Listing 2: Tablas departments, provinces y districts

```
CREATE TABLE departments (  
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
    code VARCHAR(2) UNIQUE NOT NULL,  
    name VARCHAR(100) NOT NULL,  
    is_active BOOLEAN DEFAULT TRUE  
);
```

En esta tabla provinces y districts siguen jerarqu a con claves for neas

5.2.3. M dulo Cat logo y Productos (Naranja)

Listing 3: Tabla products (Cat logo)

```
CREATE TABLE products (  
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
    category_id UUID NOT NULL REFERENCES categories(id),  
    name VARCHAR(150) NOT NULL,  
    sku VARCHAR(50) UNIQUE NOT NULL,  
    price DECIMAL(10,2) NOT NULL,  
    cost DECIMAL(10,4) NOT NULL,  
    preparation_time INTEGER,  
    is_customizable BOOLEAN DEFAULT FALSE,  
    is_active BOOLEAN DEFAULT TRUE  
);
```

Esta tabla es el coraz n del cat logo. Permite registrar bebidas, postres y sus caracter sticas (precio, tiempo de preparaci n, personalizable, etc.).

5.2.4. M dulo Inventario y Producci n (Turquesa)

Listing 4: Tabla products_ingredients

```
CREATE TABLE products_ingredients (  
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
    product_id UUID NOT NULL REFERENCES products(id) ON DELETE CASCADE,  
    ingredient_id UUID NOT NULL REFERENCES ingredients(id),  
    quantity DECIMAL(10,4) NOT NULL,  
    is_active BOOLEAN DEFAULT TRUE  
);
```

Controla las recetas de cada producto (ejemplo: cuánto café y leche lleva un Cappuccino).

5.2.5. Módulo Ventas y Pedidos (Verde)

Listing 5: Tablas orders y order_details

```
CREATE TABLE orders (  
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  user_id UUID NOT NULL REFERENCES users(id),  
  location_id UUID REFERENCES locations(id),  
  subtotal DECIMAL(10,2) NOT NULL,  
  total DECIMAL(10,2) NOT NULL,  
  payment_status VARCHAR(20),  
  order_status VARCHAR(30),  
  is_active BOOLEAN DEFAULT TRUE  
);  
  
CREATE TABLE order_details (  
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  order_id UUID NOT NULL REFERENCES orders(id) ON DELETE CASCADE,  
  product_id UUID NOT NULL REFERENCES products(id),  
  quantity INTEGER NOT NULL,  
  unit_price DECIMAL(10,2) NOT NULL  
);
```

5.2.6. Módulo Delivery y Logística (Rojo)

Listing 6: Tabla deliveries

```
CREATE TABLE deliveries (  
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  order_id UUID NOT NULL REFERENCES orders(id),  
  driver_id UUID REFERENCES drivers(id),  
  delivery_status VARCHAR(30),  
  estimated_time INTEGER,  
  delivered_at TIMESTAMP,  
  is_active BOOLEAN DEFAULT TRUE  
);
```

6. Resultados

A continuación se presentan evidencias de la implementación final de la base de datos en PostgreSQL

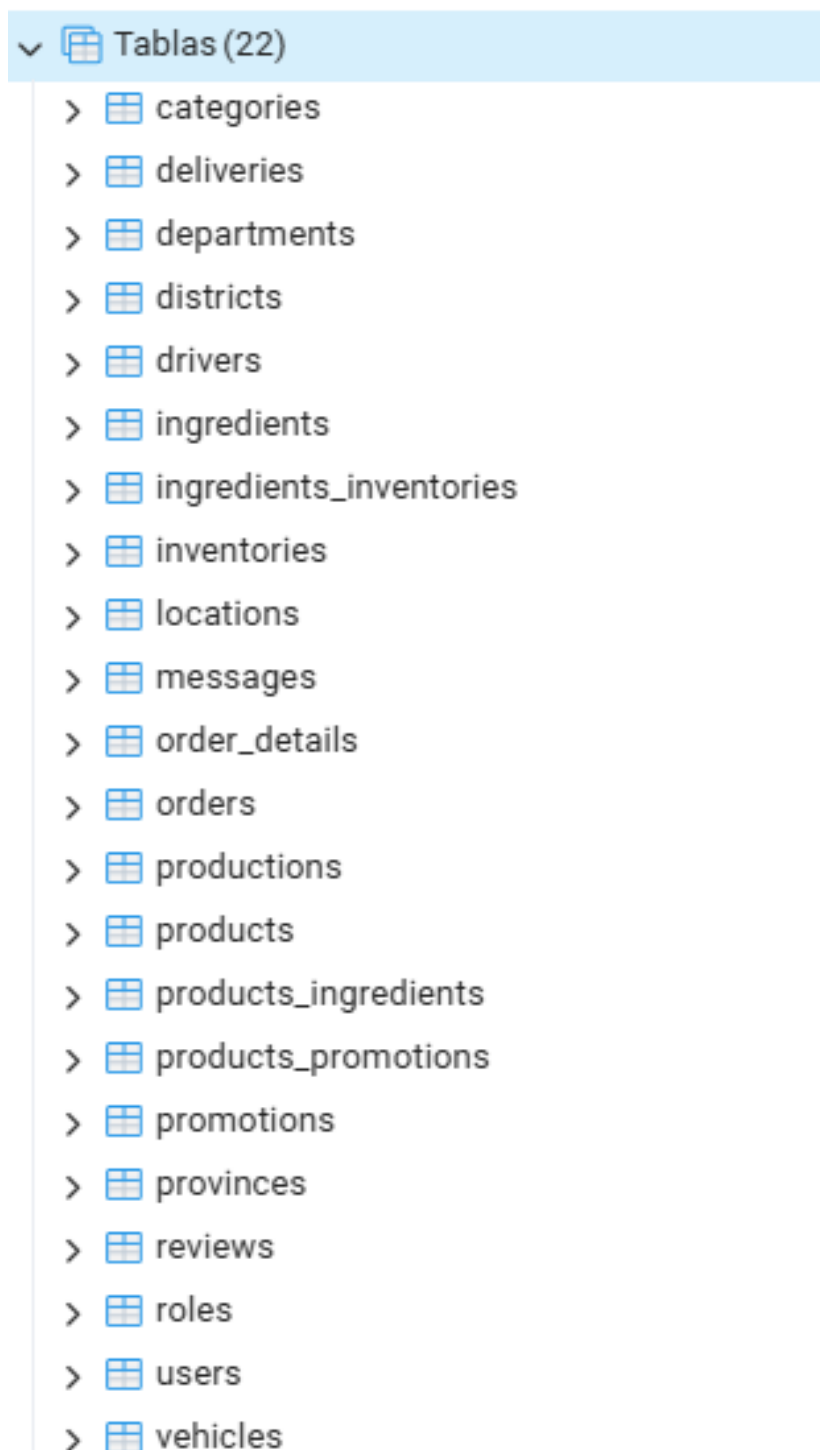


Figura 2: Listado completo de tablas creadas en el esquema public de PostgreSQL visualizado desde PGAdmin 4.

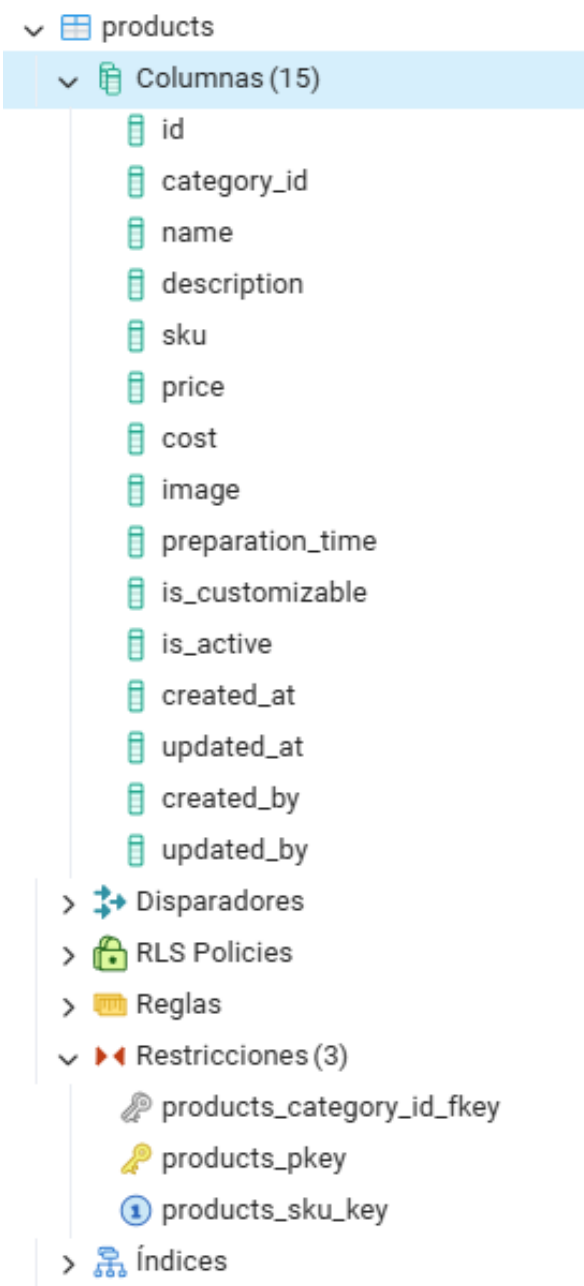


Figura 3: Estructura de la tabla **products**, mostrando los tipos de datos definidos para cada atributo, adem s se muestran las restricciones de clave for nea (Foreign Keys) definidas en la tabla.

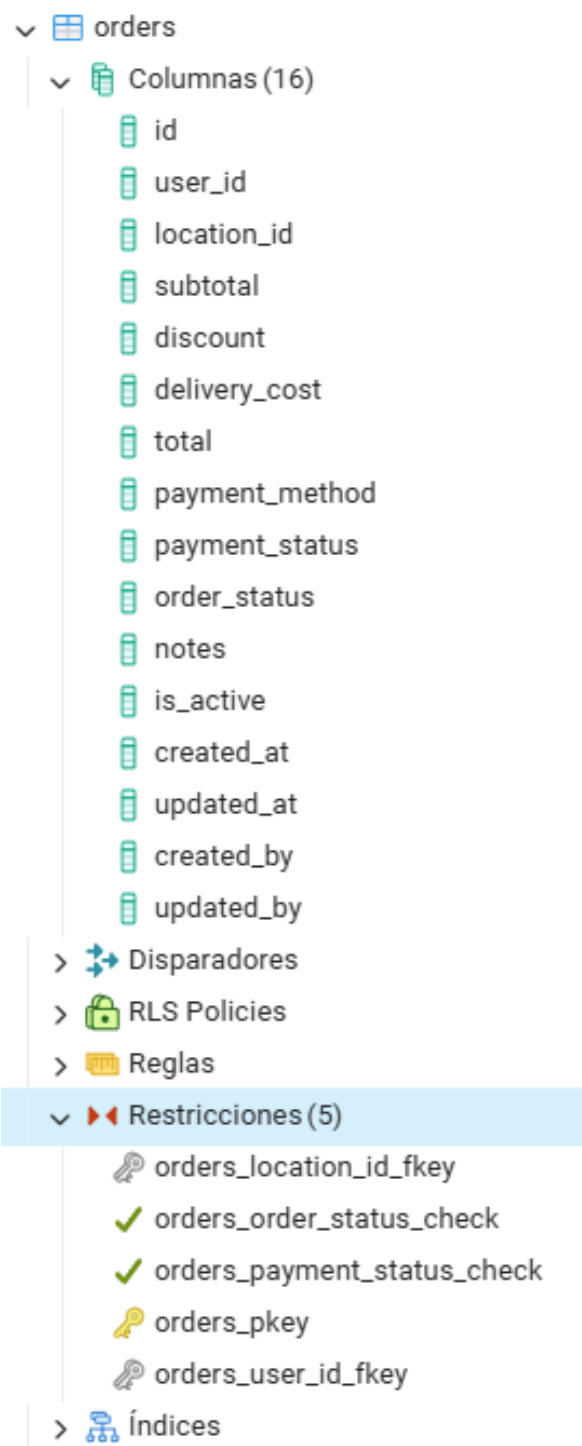


Figura 4: Estructura de la tabla pedidos evidenciando su dise o relacional, adicional se muestra las claves for neas de la tabla pedidos, mostrando su relaci n con otras entidades del sistema

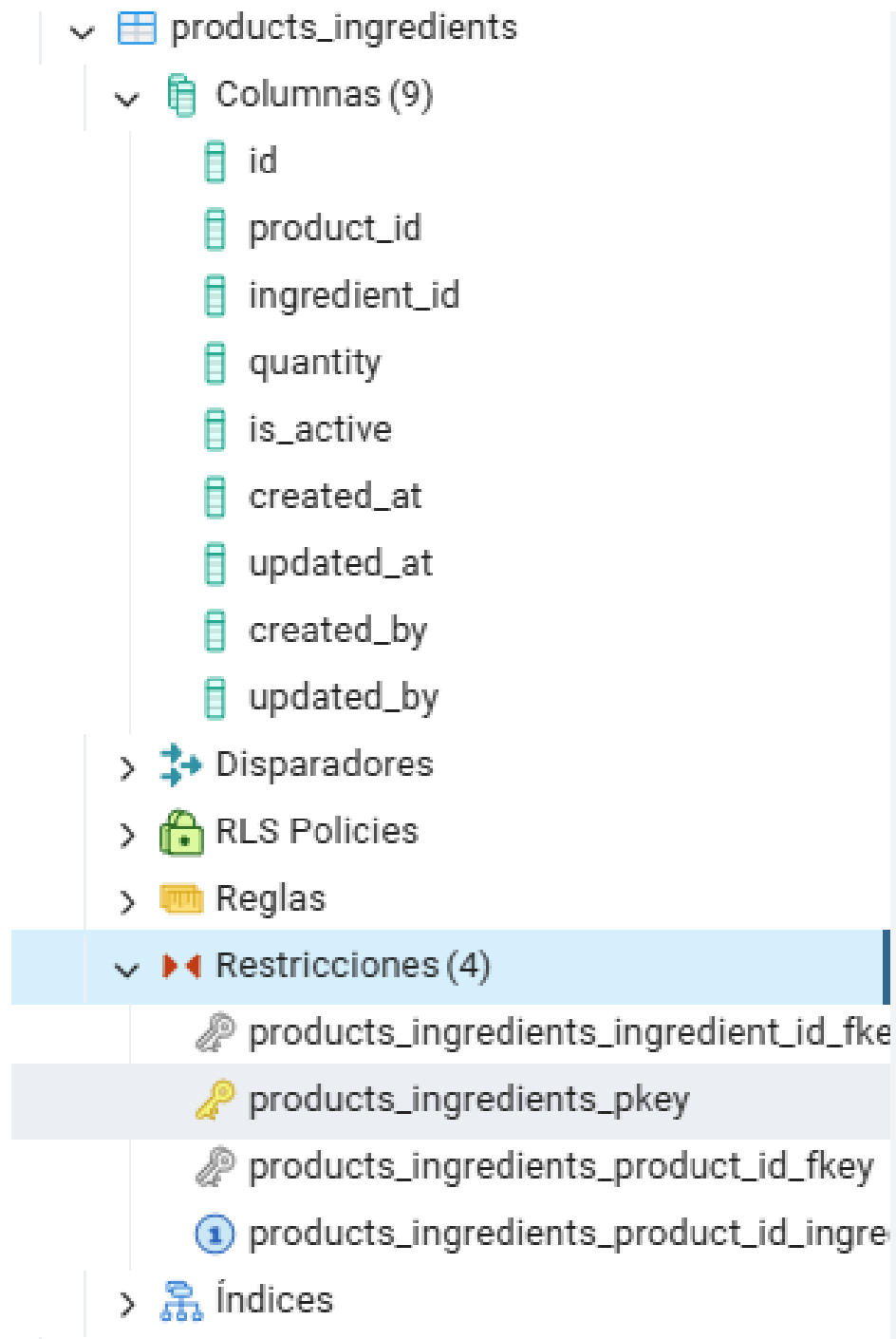


Figura 5: Tabla intermedia `products_ingredients` que resuelve la relaci n muchos a muchos entre productos e ingredientes.

- Ovid. *Database Naming Standards*. Disponible en: <https://dev.to/ovid/database-naming-standards-2061>
- Stack Overflow. *What are some of your most useful database standards?*. Disponible en: <https://stackoverflow.com/questions/976185/what-are-some-of-your-most-useful-database-standards>
- Hwa. *Database Design and Code Standard*. Disponible en: <https://medium.com/@hwa610787/database-design-and-code-standard-89c0a4beee17>

9. Rúbrica de Calificación

A continuación se presenta la rúbrica de evaluación detallada para el presente laboratorio, considerando los criterios de modelado, implementación y documentación.

Ítem	Descripción	Puntaje
DER	Elaboración del modelo lógico DER.	4
Modelo físico	Implementación del modelo físico PostgreSQL.	8
Supabase	Implementación en Supabase.	0
Informe	El laboratorio tiene un README.md que detalla toda la práctica.	3
Prueba	Se tomaron en cuenta todas las consideraciones y recomendaciones, lo que evidencia un trabajo en equipo.	0
Total		15

Cuadro 1: Rúbrica de evaluación del laboratorio.